

MECHATRONICS AND IOT

ASSIGNMENT REPORT - 5

TITLE:

Design and develop a Smart Irrigation and Water Management System. The system automates irrigation control based on environmental conditions to optimize water usage in agricultural applications.

DONE BY ,

HARI KRISHNA JAGADEESH-24MER011

HARISH DHARMALINGAM – 24MER014

RANJITH KUMAR P – 24MER048

RISHI KESAV A – 24MER049

VENKATRAMANAN B – 24MER061

Project Overview:

The Smart Irrigation and Water Management System is an IoT-based system designed to automate irrigation by continuously monitoring soil and environmental conditions. The system uses a soil moisture sensor to determine the moisture level of the soil and can also monitor parameters such as temperature and humidity using an environmental sensor.

A microcontroller processes the sensor readings and determines whether irrigation is required. Based on predefined moisture thresholds, the system automatically switches the water pump ON or OFF, supplying water only when the soil moisture level falls below the required limit.

The system can be extended with Wi-Fi connectivity to monitor soil conditions and irrigation status remotely through an IoT/cloud platform. The proposed system provides a low-cost, automated and efficient solution for optimizing water usage and improving irrigation management in agricultural applications.

Problem Statement:

Agricultural irrigation often depends on manual watering or fixed irrigation schedules, which can result in over-irrigation or under-irrigation. Excessive watering leads to water wastage, soil nutrient loss and increased energy consumption, while insufficient watering can negatively affect plant growth and crop productivity.

The lack of real-time monitoring of soil moisture and environmental conditions makes it difficult to determine the exact amount and timing of irrigation required. Therefore, there is a need for a smart and automated irrigation system that can continuously monitor soil conditions, control water supply according to the actual requirement and reduce unnecessary water consumption.

Proposed Solution:

The proposed system uses a soil moisture sensor and ESP32 microcontroller to continuously monitor the moisture content of agricultural soil. The sensor provides information about the current soil moisture condition, which is processed by the ESP32 to determine whether irrigation is required.

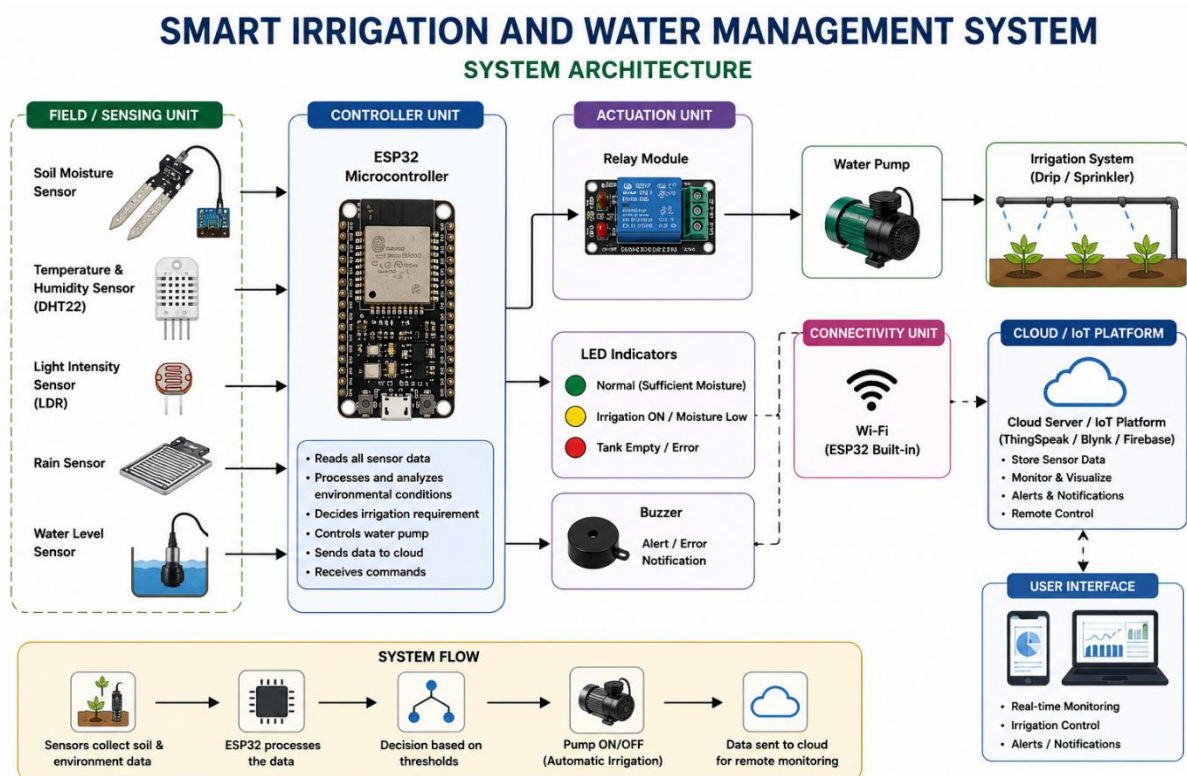
Based on predefined moisture threshold values, the system classifies the soil condition as Dry, Moist or Wet. When the soil moisture level falls below the required threshold,

the ESP32 activates a water pump through a relay module to supply water to the plants. Once the required moisture level is reached, the pump is automatically switched OFF.

Additional sensors such as a DHT11/DHT22 can be used to monitor temperature and humidity. The ESP32's Wi-Fi connectivity can also be used to transmit sensor data and irrigation status to an IoT/cloud platform for remote monitoring and analysis.

The system provides a low-cost, real-time and automated irrigation solution that reduces water wastage, minimizes manual intervention and improves water utilization for agricultural applications.

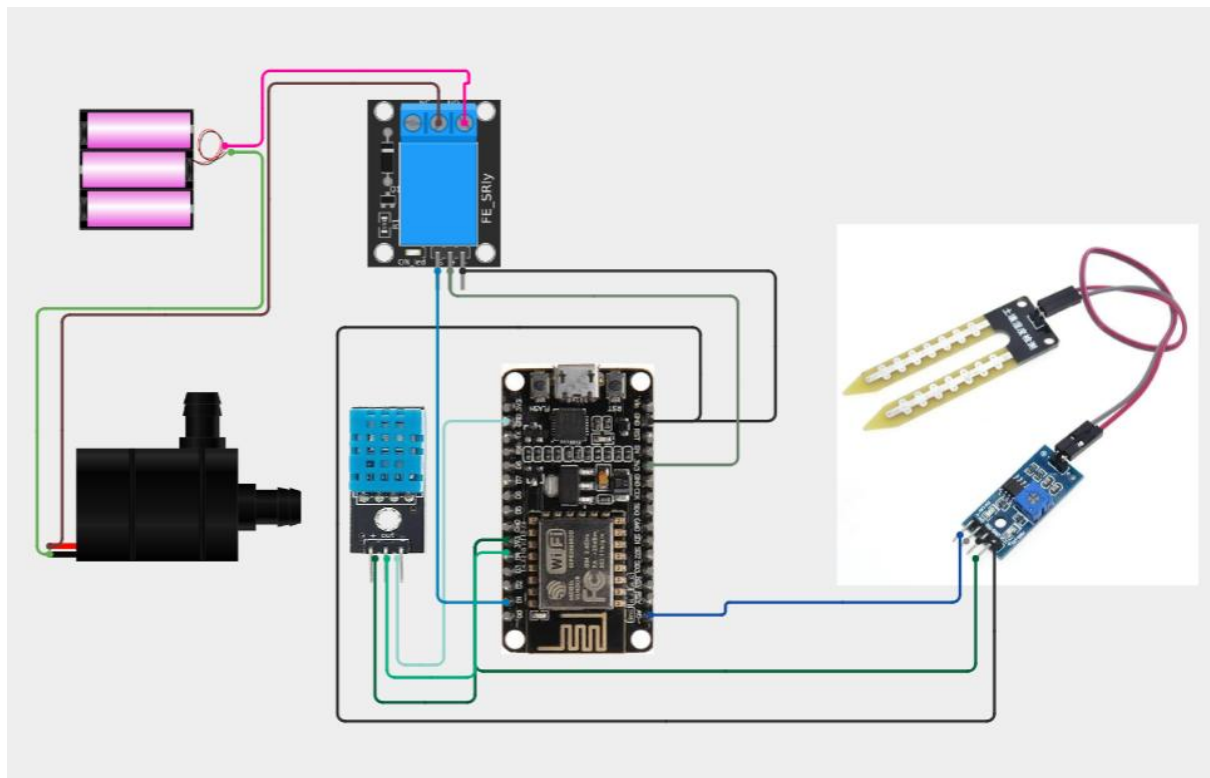
System Architecture:



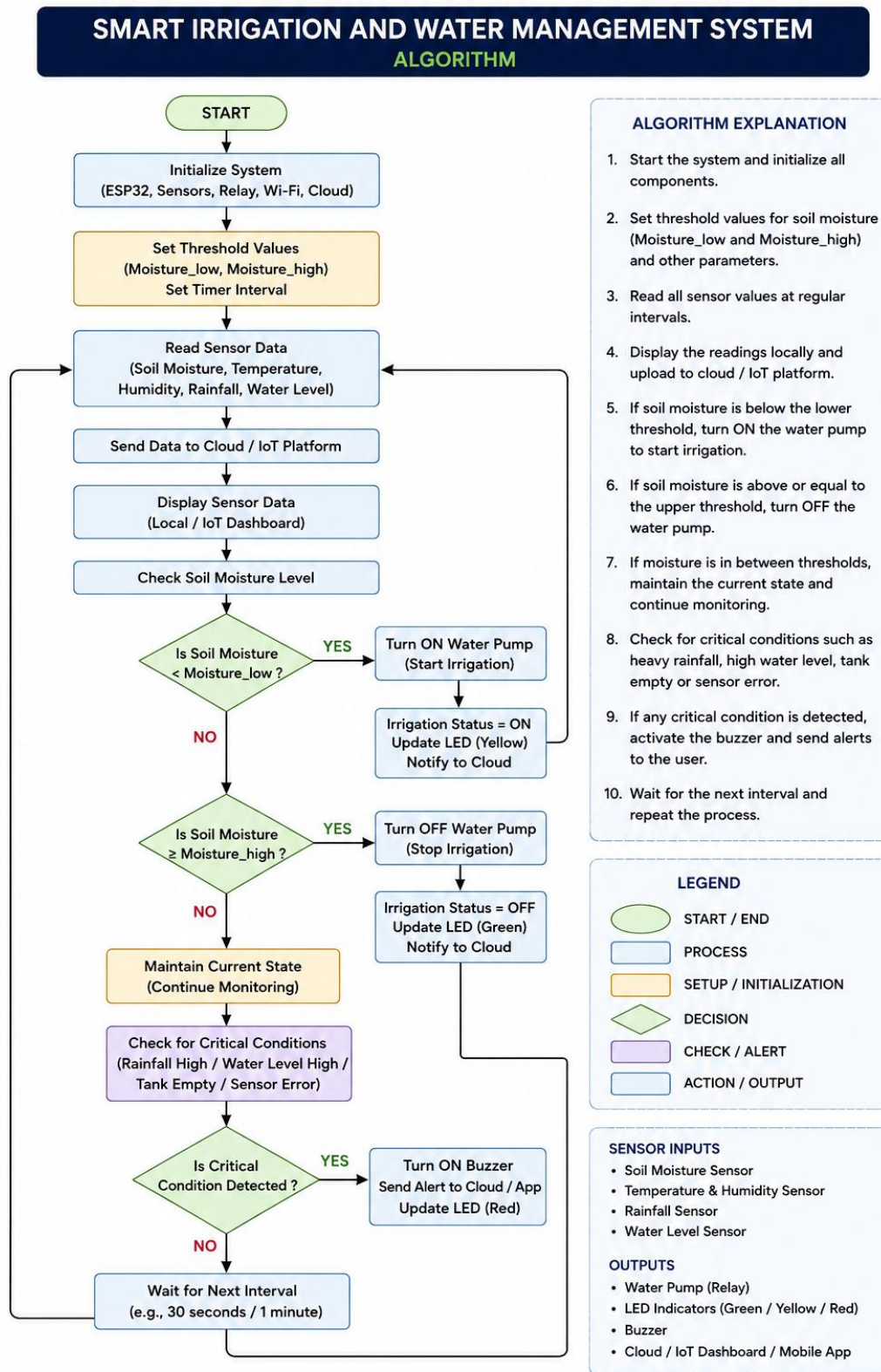
Circuit Table :

Component	Pin/Terminal	ESP8266 Connection	Purpose
Ultrasonic sensor	ECHO	D5	Receives ultrasonic pulse
Ultrasonic sensor	TRIG	D6	Sends ultrasonic pulse
LCD Display	VCC	3.3V	Safe indication
ESP8266	VIN/USB	5V USB	Hazardous indication
Buzzer	Positive (+)	GPIO 14	Alert

Circuit Design:



Algorithm:



Coding:

```
#include <ESP8266WiFi.h>

#include <ThingSpeak.h>

#include <DHT.h>

// =====

// WIFI SETTINGS

// =====

const char* ssid = "Xiaomi 11i";

const char* password = "12345678";

// =====

// THINGSPEAK SETTINGS

// =====

unsigned long channelID = 3454107;

const char* writeAPIKey = "QJL2ZTM3AYFT6N71";

WiFiClient client;

// =====

// PIN DEFINITIONS

// =====

#define SOIL_PIN A0

#define DHT_PIN 02

#define RELAY_PIN 05

#define DHT_TYPE DHT11

// =====

// DHT SETUP

// =====

DHT dht(DHT_PIN, DHT_TYPE);

// =====

// SOIL MOISTURE CALIBRATION

// =====
```

```
#define DRY_VALUE 1023

#define WET_VALUE 300

#define MOISTURE_THRESHOLD 30

// =====

// SETUP

// =====

void setup()

{

  Serial.begin(115200);

  dht.begin();

  pinMode(RELAY_PIN, OUTPUT);

  // LOW-active relay

  // Pump initially OFF

  digitalWrite(RELAY_PIN, HIGH);

  // =====

  // CONNECT TO WIFI

  // =====

  Serial.println();

  Serial.println("Connecting to WiFi...");

  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED)

  {

    delay(500);

    Serial.print(".");

  }

  Serial.println();

  Serial.println("WiFi Connected!");

  Serial.print("IP Address: ");

  Serial.println(WiFi.localIP());

  // =====
```

```
// START THINGSPEAK

// =====

ThingSpeak.begin(client);

Serial.println("ThingSpeak Started!");

}

// =====

// LOOP

// =====

void loop()

{

// =====

// READ SOIL MOISTURE

// =====

int soilRaw = analogRead(SOIL_PIN);

int soilMoisture = map(

    soilRaw,

    DRY_VALUE,

    WET_VALUE,

    0,

    100

);

soilMoisture = constrain(soilMoisture, 0, 100);

// =====

// READ DHT11

// =====

float temperature = dht.readTemperature();

float humidity = dht.readHumidity();

// =====

// CHECK DHT11

// =====
```



```
if (isnan(temperature) || isnan(humidity))
{
    Serial.println("DHT11 reading failed!");
    delay(2000);
    return;
}

// =====

// IRRIGATION CONTROL

// =====

int pumpStatus;
if (soilMoisture < MOISTURE_THRESHOLD)
{
    // Soil is dry
    digitalWrite(RELAY_PIN, LOW);
    pumpStatus = 1;
    Serial.println("Soil Status : DRY");
    Serial.println("Pump Status : ON");
}
else
{
    // Soil is sufficiently wet
    digitalWrite(RELAY_PIN, HIGH);
    pumpStatus = 0;
    Serial.println("Soil Status : MOIST");
    Serial.println("Pump Status : OFF");
}

// =====

// SERIAL MONITOR

// =====
```

```
Serial.println("-----");
Serial.print("Soil Raw Value : ");
Serial.println(soilRaw);
Serial.print("Soil Moisture : ");
Serial.print(soilMoisture);
Serial.println(" %");
Serial.print("Temperature  : ");
Serial.print(temperature);
Serial.println(" °C");
Serial.print("Humidity    : ");
Serial.print(humidity);
Serial.println(" %");
Serial.print("Pump Status  : ");
Serial.println(pumpStatus);
// =====
// SEND DATA TO THINGSPEAK
// =====
ThingSpeak.setField(1, soilMoisture);
ThingSpeak.setField(2, temperature);
ThingSpeak.setField(3, humidity);
ThingSpeak.setField(4, pumpStatus);
// Optional status message
if (pumpStatus == 1)
{
    ThingSpeak.setStatus("Automatic irrigation - Pump ON");
}
else
{
    ThingSpeak.setStatus("Automatic irrigation - Pump OFF");
}
```

```

// Send all fields
int response = ThingSpeak.writeFields(
    channelID,
    writeAPIKey
);
if (response == 200)
{
    Serial.println("ThingSpeak update successful!");
}
else
{
    Serial.print("ThingSpeak update failed. HTTP error: ");
    Serial.println(response);
}
Serial.println("-----");
// ThingSpeak update interval
// Keep at least 15 seconds between updates
delay(20000);
}

```

System Working:

- The Smart Irrigation and Water Management System uses a soil moisture sensor and ESP32 microcontroller to continuously monitor the moisture level of agricultural soil. Additional sensors such as a DHT11/DHT22 can be used to monitor temperature and humidity.
- The soil moisture sensor detects the moisture content of the soil and sends the reading to the ESP32. The ESP32 processes the sensor data and compares the moisture level with predefined threshold values.
- When the soil moisture level falls below the minimum threshold, the ESP32 activates a relay module, which switches the water pump ON to provide irrigation. Once the soil

reaches the required moisture level, the ESP32 switches the pump OFF automatically, preventing unnecessary water usage.

- The sensor readings and irrigation status can be displayed locally using an OLED display. The ESP32 can also use its built-in Wi-Fi connectivity to transmit the data to an IoT/cloud platform for remote monitoring and analysis.
- A buzzer or notification system can be used to alert the user during abnormal conditions such as low water availability, sensor failure or other critical conditions. The system continuously repeats the monitoring and irrigation process, providing an automated and water-efficient irrigation solution for agricultural applications.

Bill Of Material:

S. no	Component	Specification	Quantity	Cost
1	ESP8266	Node MCU ESP8266	1	Rs. 450
2	Soil moisture sensor	Capacitive/Resistive Soil Moisture Sensor	1	Rs. 70
3	Relay module	1-Channel 5V Relay	1	Rs. 150
5	Mini water pump	DC Mini Water Pump	1	Rs. 100
6	DhT11 sensor	Temperature & Humidity Sensor	1 set	Rs. 50
7	Breadboard	Standard 830-point	Total	Rs. 820
8	Jumper wires	Male-Male/Male-Female		

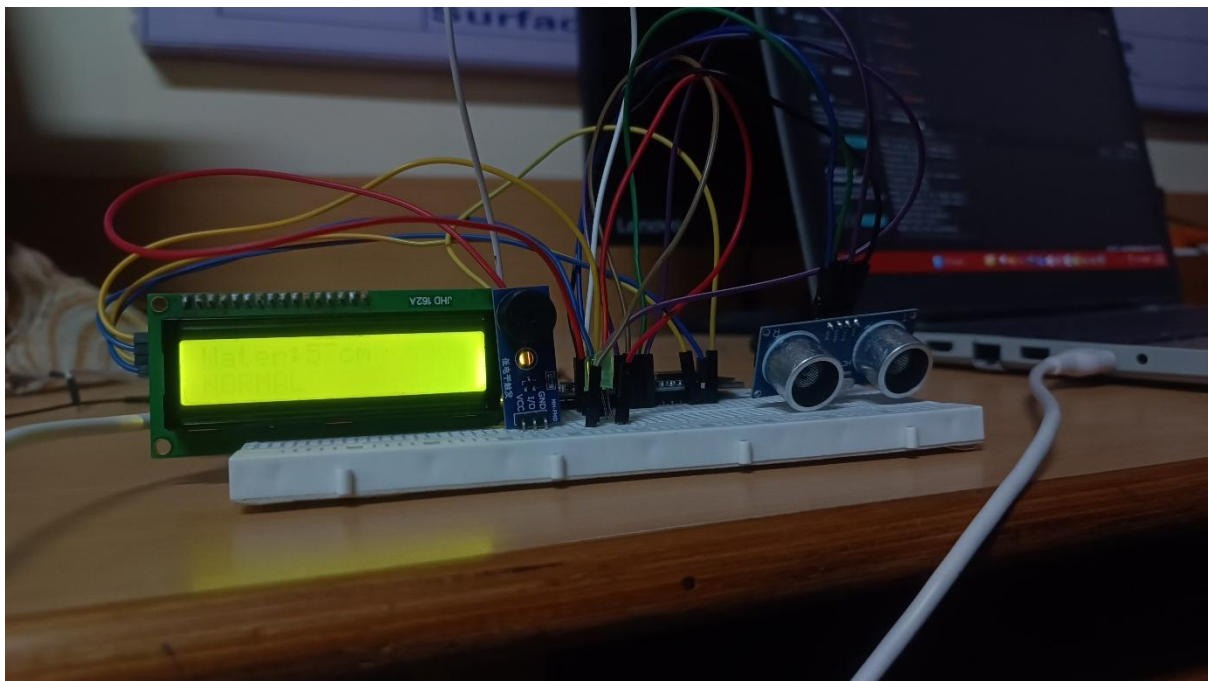
Prototype Implementation:

- The prototype was implemented using an ESP32 microcontroller, soil moisture sensor, relay module, water pump, OLED display and buzzer. The soil moisture sensor was inserted into the soil to continuously measure the moisture level and send the readings to the ESP32.
- The ESP32 processes the sensor data and compares the measured moisture level with predefined threshold values. When the soil becomes dry and the moisture level falls below the required limit, the ESP32 activates the relay module, which switches the water pump ON and supplies water to the plants. Once the required moisture level is reached, the pump is automatically switched OFF.

- The OLED display shows the current soil moisture level and irrigation status, while the buzzer can provide alerts during abnormal conditions such as low water availability or sensor failure. The ESP32's built-in Wi-Fi capability can also be used to transmit sensor readings to an IoT/cloud platform for remote monitoring.
- The prototype follows the data flow:

Soil Moisture Sensor → ESP32 → Decision Based on Moisture Level → Relay → Water Pump → Irrigation

The implemented prototype demonstrates a low-cost, automated and water-efficient irrigation system that reduces manual intervention and optimizes water usage in agricultural applications.



Results and Performance:

- The developed Smart Irrigation and Water Management System successfully monitored soil moisture and automatically controlled the irrigation process using the ESP32, soil moisture sensor, relay module and water pump. The system responded to changes in soil moisture and supplied water only when the soil became sufficiently dry.
- When the soil moisture level fell below the predefined threshold, the ESP32 activated the relay and switched the water pump ON. Once the required moisture level was

reached, the pump was automatically switched OFF. This demonstrated effective automatic irrigation and reduced the need for manual control.

- The OLED display provided real-time information about the soil moisture level and irrigation status. The system can also be connected to a cloud platform through the ESP32's Wi-Fi capability for remote monitoring and analysis.

Performance Analysis

Parameter	Result
Soil Moisture Monitoring	Successfully measured
Automatic Irrigation	Successfully implemented
Pump Control	Automatic ON/OFF
Local Monitoring	OLED display
Water Usage	Reduced through threshold-based irrigation
Response	Real-time
Manual Intervention	Minimized
Remote Monitoring	Wi-Fi/IoT integration supported

Overall, the prototype demonstrated a **reliable, low-cost and water-efficient irrigation system**. By supplying water only when required, the system helps reduce water wastage and improves irrigation efficiency. The system can be further enhanced with weather monitoring, water-level sensing, flow measurement and cloud-based alerts for large-scale agricultural applications.

Channel Stats

Created: 5 days ago

Last entry: less than a minute ago

Entries: 5

